Recommender Systems Project Report

on

TopicLens: An Explainable News Recommender

Submitted to Manipal University Jaipur

Towards the partial fulfilment for the Award of the Degree of

B. Tech Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

2026–2027

By

Riddhima
2427010547



MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

Pranshu Pranjal

Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur
Jaipur, Rajasthan

ABSTRACT

News sites decide what millions of people read each day, but they rarely say why a story was picked. Most of them use models that are hard to inspect, so a reader cannot tell whether a recommendation came from their own history, from what is popular, or from something else. Our objective was to build a news recommender that ranks stories well using a reader's click history, chosen topics, reactions to earlier recommendations and direct feedback, and that can show the exact reasons behind each pick.

We used MIND-small, a public dataset of real, anonymised Microsoft News logs with 65,238 articles, 94,057 users and 8.58 million candidate rows. From the article text we learned 30 topics with non-negative matrix factorisation and a 100-dimensional LSA text space, and built ten features for each user and article pair. We trained and compared fourteen rankers on the same split: simple rules, a KNN classifier, linear and kernel SVMs, a random forest, logistic regression, LightGBM LambdaRank, an NRMS-style neural model trained on a GPU, and a hybrid of LightGBM and the neural score.

On 73,152 test visits that the models never saw, the hybrid did best with an AUC of 66.60 and nDCG@10 of 41.64. Logistic regression reached 63.84 AUC, level with LightGBM, the linear SVM and slightly above the neural model, while KNN reached only 55.66. The published BATM-NR model scores 68.13 AUC on the same data, so our best model is about 1.5 points behind it while using only titles and abstracts.

We built a web application, TopicLens, around the logistic regression because its score splits exactly into per-feature parts. Each recommendation shows a short reason and an optional breakdown, and readers can steer the feed with topic sliders, mutes and "more/less like this". Accounts, preferences and every click are stored in a Supabase Postgres database with row-level security. The results suggest that a simple, fully explainable model loses little accuracy against stronger black-box rankers on this task.

CONTENTS

Abstract **i**

1 Introduction **1**
1.1 Motivation 1
1.2 Project Statement and Objectives 1
1.3 Organisation of the Report 2

2 Background Material **3**
2.1 Conceptual Overview 3
2.1.1 News recommendation as click prediction 3
2.1.2 Text representations 3
2.1.3 Models compared 3
2.1.4 Evaluation metrics 4
2.2 Technologies Involved 4

3 Methodology **5**
3.1 Dataset 5
3.2 Train, Validation and Test Split 5
3.3 Topic and Text Models 5
3.4 Features 6
3.5 Models and Training 6
3.6 How the Application Recommends 7
3.7 System Architecture 8

4 Results and Conclusions **9**
4.1 Results 9
4.2 Discussion 9
4.3 Limitations 10
4.4 Conclusions 10

References **11**

CHAPTER 1
INTRODUCTION

1.1 Motivation

Most people now get their news through a feed. The order of that feed is set by a recommender system that has learned from what readers clicked in the past. These systems are good at ranking, but they are opaque. A reader sees a story near the top and has no way of knowing whether it is there because of their own reading, because it is popular, or because it is new. If the feed goes wrong, for example by showing too much of one topic, there is usually no direct way to correct it.

Research on explainable recommendation tries to fix this. Liu et al. [1] proposed BATM-NR, a neural news recommender that learns topics inside the model and uses them to explain its picks. Their work was our starting point. We wanted to see how far a simpler approach could go: learn topics from the text, feed them into a trained click model whose output can be split exactly into reasons, and give the reader controls that change the ranking straight away.

Applications of this kind of system include news portals, content apps and any feed where trust matters. The main advantage of an exact explanation is that it cannot disagree with the model, because it *is* the model's score written out term by term.

1.2 Project Statement and Objectives

The project statement is: *recommend the most relevant news articles to each user from their reading and click history, their preferred topics, their interaction with earlier recommendations and their explicit feedback, and explain every recommendation.* Ranking comes first; the explanations support it.

The objectives were:

1. Use a real, public click dataset rather than synthetic data.
2. Train real machine learning models, including KNN and SVM, and compare them on one fixed train and test split with standard ranking metrics.
3. Pick a model for the application that is accurate and whose score can be explained exactly.
4. Use four kinds of user signal: click history, chosen sections, skipped recommendations and explicit feedback.

5. Build a working web application with sign-in and a database that stores each user's profile and interactions.

1.3 Organisation of the Report

Chapter 2 covers the background: the recommendation problem, the models we used and the tools. Chapter 3 describes the dataset, the split, the features, the training of each model and the design of the application. Chapter 4 presents the results, discusses them and lists the limits of the work and what we would do next.

CHAPTER 2

BACKGROUND MATERIAL

2.1 Conceptual Overview

2.1.1 News recommendation as click prediction

In a news log, each *impression* is one visit: a user, the list of articles they had clicked before, and a set of candidate articles shown to them, each marked clicked or not. A recommender gives every candidate a score, and the candidates are sorted by that score. A model is good if clicked articles end up near the top. This makes the task a mix of binary classification (click or not) and ranking within each impression.

2.1.2 Text representations

TF-IDF turns a headline and abstract into a sparse vector of word weights. **Latent semantic analysis (LSA)** applies a truncated SVD to the TF-IDF matrix and gives a dense vector in which similar articles lie close together [2]. **Non-negative matrix factorisation (NMF)** splits the TF-IDF matrix into a document-topic matrix and a topic-word matrix with no negative entries [3]. Because the weights are non-negative, each topic can be read as a list of words, which is why we use it for explanations.

2.1.3 Models compared

- **K-nearest neighbours (KNN)** predicts a click from the labels of the most similar training rows in feature space.
- **Support vector machines (SVM)** find a maximum-margin boundary. We used a linear SVM and an RBF kernel SVM, with the kernel approximated by the Nyström method so it fits in memory [4].
- **Random forest** averages many decision trees trained on bootstrap samples [5].
- **Logistic regression** models the click probability as $\sigma(b + \sum_j w_j z_j)$, where z_j are standardised features. Each term $w_j z_j$ is that feature's exact contribution.
- **LightGBM LambdaRank** is a gradient-boosted tree model trained directly to improve the ranking metric nDCG [6, 7].
- **NRMS** is a neural recommender that encodes each article and then encodes the user with self-attention over their clicked articles [8]. The click score is the dot product of the two vectors.

2.1.4 Evaluation metrics

We used the four metrics from the MIND benchmark [9], each computed per impression and then averaged:

- **AUC**: the chance that a random clicked article is ranked above a random unclicked one in the same impression. 50 is a coin flip.
- **MRR**: the mean of $1/\text{rank}$ of the clicked articles, which rewards putting a click near the top.
- **nDCG@5 and nDCG@10**: how much of the ideal ranking quality is reached in the top 5 or 10 positions, with lower positions discounted.

2.2 Technologies Involved

- **Python** with pandas, NumPy and scikit-learn for data processing, topic models and the classic models [10]; LightGBM for gradient boosting [6].
- **PyTorch** with CUDA and sentence-transformers (MiniLM) for the neural model, trained on an NVIDIA RTX 3050 laptop GPU with 4 GB of memory.
- **React, TypeScript and Vite** for the web application. The trained logistic regression runs in the browser.
- **Supabase** (hosted PostgreSQL with authentication) for accounts and stored user data.
- **Git and GitHub** for version control, with each feature committed separately.

CHAPTER 3

METHODOLOGY

3.1 Dataset

We used MIND-small [9, 11], released by Microsoft Research for non-commercial research. It contains logs from Microsoft News collected in November 2019. User ids are anonymised. Table 3.1 gives the sizes.

Table 3.1: MIND-small at a glance

Item	Count
News articles (title, abstract, category, subcategory)	65,238
Users	94,057
Training impressions (9 to 14 Nov 2019)	156,965
Test impressions (15 Nov 2019)	73,152
Candidate rows (user, article, clicked or not)	8,584,442
Click rate	4.05%
Categories	17

Only about one candidate in 25 is clicked, so the classes are very unbalanced. This is one reason we report ranking metrics instead of plain accuracy: a model that predicts "no click" for everything would be 96% accurate and useless.

3.2 Train, Validation and Test Split

We split by time so that no model sees the future:

- **Fit set**: training impressions from 9 to 13 November (126,695 impressions).
- **Validation set**: training impressions from 14 November (30,270 impressions), used for early stopping and for picking the best epoch.
- **Test set**: the official MIND-small dev set from 15 November (73,152 impressions). It was used once per model, at the end.

Every model in Table 4.1 was trained on the fit set and scored on the same test set. 95% confidence intervals come from 500 bootstrap resamples of the test impressions and are about ±0.2 points for every metric.

3.3 Topic and Text Models

Each article's title and abstract were turned into TF-IDF vectors. We then fitted:

- NMF with $K = 30$ topics, the value Liu et al. found best [1]. Each topic is described by its top eight words and was given a short human-readable name.
- LSA with 100 dimensions, used to measure how close an article's wording is to what the user has read.

3.4 Features

For every (user, candidate article) pair we computed ten features. Popularity features only use clicks from days *before* the impression, so they do not leak the label.

1. **Topic match**: similarity between the article's topic mix and the user's history.
2. **Text similarity**: LSA similarity between the article and the user's history.
3. **Category affinity**: share of the user's reads in the article's category.
4. **Subcategory affinity**: the same at subcategory level.
5. **Prior clicks**: clicks the article received on earlier days.
6. **Prior CTR**: click-through rate on earlier days.
7. **Exposure**: how many times it had been shown before.
8. **Freshness**: hours since the article first appeared.
9. **History length**: how many articles the user had read.
10. **Impression size**: how many articles were shown together.

3.5 Models and Training

- **Rule baselines**: random order, recent exposure, topic match only, text similarity only, popularity only and category match only. These show how much each signal is worth alone.
- **KNN classifier** with $k = 50$ and distance weighting, fitted on 60,000 sampled rows.
- **Linear SVM** ($C = 0.05$, balanced class weights) on 400,000 sampled rows, and **RBF kernel SVM** using 300 Nyström components on 150,000 sampled rows.
- **Random forest** with 200 trees of depth up to 14, on 400,000 sampled rows.

KNN and kernel methods do not scale to the full two million fit rows on a laptop, so these four models were fitted on random samples of the fit set. All of them were scored on every test row, and all use the same standardised features.

- **Logistic regression** on standardised features. This is the model the application uses.

- **LightGBM LambdaRank**, early-stopped on the validation day. The best iteration was 273.
- **Neural NRMS-style model**. Articles are encoded with a MiniLM sentence embedding plus a category embedding; the user encoder applies multi-head self-attention and additive attention over the last 50 reads. It was trained with negative sampling (one click against four unclicked articles from the same impression) on 189,519 clicks for 4 epochs, which took about 8 minutes on the GPU. Validation AUC went 63.39, 63.39, 63.47, 63.50 over the four epochs and the last epoch was kept.
- **Hybrid**: LightGBM with the neural score added as an eleventh feature (best iteration 227).

3.6 How the Application Recommends

The application does not use KNN or SVM. They were trained for comparison. It ranks with the trained logistic regression, because its score is a plain sum:

$$s(u, a) = b + \sum_{j=1}^{10} w_j \, z_j(u, a), \tag{3.1}$$

where z_j is the j-th standardised feature for user u and article a. The application shows each term $w_j z_j$ as that reason's share of the score. The topic and text terms are further split across the history articles that produced them, so the reader sees which past reads led to a pick. Unit tests check that the parts add up to the score.

The four kinds of signal from the project statement enter the score as follows:

1. **Reading and click history**: opening a story counts as a click and is added to the history, so all history-based features update at once.
2. **Preferred sections**: sections chosen at sign-up add +2.0 to the log-odds of articles in those sections, shown as the reason "a section you chose".
3. **Interaction with earlier recommendations**: when a reader clicks a story, the stories ranked above it that they passed over get a penalty of 0.5 per pass, up to three passes. This is the usual skip-above reading of click logs.
4. **Explicit feedback**: "More like this" and "Less like this", "Not interested", topic sliders from 0× to 2× and topic mutes. Each change is logged with its effect on the top ten.

3.7 System Architecture

Figure 3.1 shows the parts of the system. Training happens offline in Python. The training script exports the article data, the topic model and the logistic regression weights to JSON files, which the browser application loads. Sign-in and storage go through Supabase.

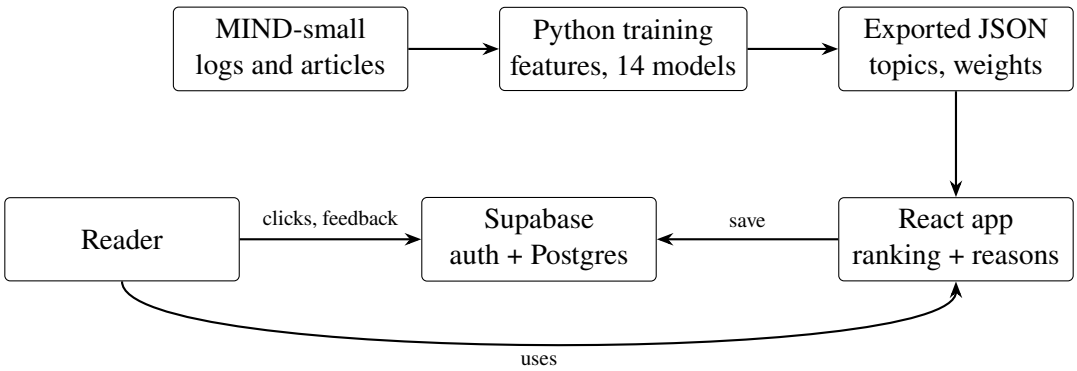


Figure 3.1: System architecture of TopicLens

The database has three tables:

- `profiles`: one row per user with display name, chosen sections and starting history.
- `preferences`: topic sliders, mutes, liked, disliked and dismissed articles, and skip counts.
- `interactions`: an append-only log of every click, read, like, dislike, dismiss and skip, with its time and rank.

Row-level security lets each user read and write only their own rows. Users can add to the interaction log but cannot edit or delete it. A trigger creates the profile and preference rows when someone signs up. Sign-in supports email and password, Google and anonymous guests.

CHAPTER 4

RESULTS AND CONCLUSIONS

4.1 Results

Table 4.1 lists every model on the 73,152 test impressions, sorted by AUC. The last row is copied from the BATM-NR paper [1] for reference; we did not re-run it.

Table 4.1: Ranking results on the MIND-small test set (all values ×100)

Model	AUC	MRR	nDCG@5	nDCG@10
Random	50.07	21.91	22.38	28.66
Recent exposure	51.69	23.00	24.40	30.47
Topic match (NMF)	51.83	22.34	22.41	29.35
KNN classifier ($k = 50$)	55.66	25.97	27.85	34.00
Text similarity (LSA)	56.03	25.61	26.89	33.39
Popularity (earlier days)	58.61	27.24	28.66	34.73
Kernel SVM (RBF, Nyström)	60.40	27.86	30.27	36.82
Category match	60.94	29.24	31.71	37.64
Random forest	61.79	29.26	31.95	38.15
Neural NRMS (GPU)	63.14	29.75	32.20	38.64
Linear SVM	63.74	30.56	33.52	39.59
LightGBM LambdaRank	63.83	30.58	33.26	39.46
Logistic regression (used in app)	63.84	30.56	33.54	39.61
Hybrid: LightGBM + neural score	**66.60**	**32.05**	**35.47**	**41.64**
BATM-NR (from [1], not re-run)	68.13	32.62	36.36	42.46

The features the trees used most were earlier-day CTR (15% of total gain), article category (12%), subcategory affinity (12%), freshness (10%) and text similarity (10%).

4.2 Discussion

Single signals are weak. Topic match alone barely beats random (51.83), and text similarity alone reaches 56.03. Category match is the strongest single rule at 60.94. The trained models gain about three AUC points over that by combining the signals.

Linear models are as good as the trees here. Logistic regression, the linear SVM and LightGBM all land within 0.1 AUC of each other, which is inside the confidence interval. With these ten features, most of the useful signal seems to be roughly linear. This is what made logistic regression the right choice for the application: we get the exact explanation for free.

KNN and kernel SVM do worse. KNN reached 55.66. With only 4% positives, most of a row's 50 neighbours are likely to be non-clicks, which makes its scores coarse, and it saw only 60,000 training rows. The kernel SVM (60.40) used an approximated kernel and a smaller sample than the linear SVM, and it fell below its linear version.

The neural model helps most as a feature. On its own the NRMS-style model scored 63.14, a little below logistic regression. Added as a feature to LightGBM, it lifted AUC by 2.8 points to 66.60, our best result. The neural model reads the wording of the headlines, and the trees add popularity and freshness, which the neural model does not see. The hybrid is reported but not used in the app, because it needs the neural encoder at run time and its scores cannot be split exactly.

Gap to the paper. Our best model is 1.53 AUC below BATM-NR. We used only titles and abstracts and a frozen sentence encoder, while BATM-NR trains its text encoder end to end.

4.3 Limitations

- We have not run a user study, so we cannot claim that the explanations or controls improve trust or understanding. This is still a hypothesis.
- MIND covers six days in 2019. In the app the "news" is from that period, and popularity and freshness are computed relative to it.
- The section boost (+2.0) and skip-above penalty (0.5) were set by hand, not learned from data.

4.4 Conclusions

We built TopicLens, a news recommender trained on real Microsoft News logs that uses click history, chosen sections, skipped recommendations and explicit feedback, and explains every pick exactly. Out of fourteen rankers tested on the same 73,152 unseen visits, the hybrid of LightGBM and a GPU-trained neural model was the most accurate (66.60 AUC). Logistic regression scored 63.84, matching LightGBM and the linear SVM, and we chose it for the application because its score breaks down into reasons with no approximation. KNN and the kernel SVM were clearly weaker on this data.

For future work, we plan to run the user study described in our evaluation plan, learn the feedback weights from logged interactions instead of setting them by hand, and test whether a distilled version of the hybrid can keep exact explanations while closing part of the gap to BATM-NR.

REFERENCES

[1] D. Liu, D. Greene, I. Li, X. Jiang and R. Dong, "Topic-Centric Explanations for News Recommendation", *ACM Transactions on Recommender Systems*, vol. 3, no. 2, 2024, doi:10.1145/3680295.

[2] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer and R. Harshman, "Indexing by Latent Semantic Analysis", *Journal of the American Society for Information Science*, vol. 41, no. 6, 1990, pp. 391–407.

[3] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization", *Nature*, vol. 401, 1999, pp. 788–791.

[4] C. K. I. Williams and M. Seeger, "Using the Nyström Method to Speed Up Kernel Machines", in *Advances in Neural Information Processing Systems 13*, 2001.

[5] L. Breiman, "Random Forests", *Machine Learning*, vol. 45, no. 1, 2001, pp. 5–32.

[6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", in *Advances in Neural Information Processing Systems 30*, 2017.

[7] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An Overview", Microsoft Research Technical Report MSR-TR-2010-82, 2010.

[8] C. Wu, F. Wu, S. Ge, T. Qi, Y. Huang and X. Xie, "Neural News Recommendation with Multi-Head Self-Attention", in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 6389–6394.

[9] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu and M. Zhou, "MIND: A Large-scale Dataset for News Recommendation", in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 3597–3606.

[10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.

[11] MIND: Microsoft News Recommendation Dataset, msnews.github.io, last accessed 30 October 2026.